

## 1.2 넘파이의 특징

In [ ]:

```
# 리스트 : 0 ~ 10까지 2만큼 증가하는 항목을 갖는 리스트
n = list(range(0,11,2))
n
```

Out[2]:

```
[0, 2, 4, 6, 8, 10]
```

In [ ]:

```
n = list(range(0,1,0.2))
n
```

```
-----
TypeError                                Traceback (most recent call
last)
<ipython-input-1-7efb92e94921> in <module>()
----> 1 n = list(range(0,1,0.2))
      2 n
```

TypeError: 'float' object cannot be interpreted as an integer

In [ ]:

```
import numpy as np

arr = np.arange(0,11,2)
arr
```

Out[2]:

```
array([ 0,  2,  4,  6,  8, 10])
```

In [ ]:

```
import numpy as np

arr = np.arange(0, 1.1, 0.2)
arr
```

Out[4]:

```
array([0. , 0.2, 0.4, 0.6, 0.8, 1. ])
```

In [ ]:

```
# 0 ~ 10 까지 2씩 증가하는 리스트 n 생성
n = list(range(0,11,2))

# n의 각 항목을 2만큼 증가
for i in range(len(n)):
    n[i] = n[i] + 2

n
```

Out[13]:

[2, 4, 6, 8, 10, 12]

In [ ]:

```
import numpy as np

# 0부터 1까지 0.2씩 증가하는 항목을 갖는 배열
arr1 = np.arange(0,1.1,0.2)
# 6개의 1을 갖는 배열
arr2 = np.ones(6)

# arr1 배열 항목 전체를 0.1만큼 증가한 값
print(arr1 + 0.1)
# arr1 배열과 arr2 배열에서 같은 위치 항목끼리의 덧셈
print(arr1 + arr2)
```

[0.1 0.3 0.5 0.7 0.9 1.1]  
[1. 1.2 1.4 1.6 1.8 2. ]

In [ ]:

```
import numpy as np

arr = np.arange(0, 1.1, 0.2)
arr = arr + 0.1
print('합: ', np.sum(arr))
print('평균: ', np.mean(arr))
print('최대: ', np.max(arr))
print('최소: ', np.min(arr))
print('분산: ', np.var(arr))
print('표준편차:', np.std(arr))
```

합: 3.6  
평균: 0.6  
최대: 1.1  
최소: 0.1  
분산: 0.11666666666666667  
표준편차: 0.34156502553198664

## 2.1 넘파이 배열 생성하기

In [ ]:

```
import numpy as np

arr1 = np.array([1,2,3])
arr2 = np.array((1.2, 2.3, 3.5))
arr3 = np.array([[1,2,3],[4,5,6]])

print(arr1)
print(arr2)
print(arr3)
```

```
[1 2 3]
[1.2 2.3 3.5]
[[1 2 3]
 [4 5 6]]
```

In [ ]:

```
import numpy as np

arr1 = np.arange(1,10)
arr2 = np.arange(1,10,2)
arr3 = np.linspace(0,0.9,4)

print(arr1)
print(arr2)
print(arr3)
```

```
[1 2 3 4 5 6 7 8 9]
[1 3 5 7 9]
[0.  0.3 0.6 0.9]
```

In [ ]:

```
import numpy as np

arr1 = np.zeros(3)
arr2 = np.zeros([2,3], dtype=int)
arr3 = np.ones([2,2])
arr4 = np.empty(2)

print(arr1)
print(arr2)
print(arr3)
print(arr4)
```

```
[0. 0. 0.]
[[0 0 0]
 [0 0 0]]
[[1. 1.]
 [1. 1.]]
[2.43266066e-316 2.12199579e-314]
```

In [ ]:

```
import numpy as np

arr1 = np.random.rand(2)
arr2 = np.random.randn(2)
arr3 = np.random.randint(1,10,size=3)
arr4 = np.random.randint(10,size=(2,2))

print(arr1)
print(arr2)
print(arr3)
print(arr4)
```

```
[0.14842125 0.9951446 ]
[-0.7420922 -0.58571473]
[5 6 3]
[[0 2]
 [3 3]]
```

## 2.2 배열의 속성 출력

In [ ]:

```
import numpy as np

arr1 = np.ones([3,4])

print(arr1)
print('size :', arr1.size)
print('dtype :', arr1.dtype)
print('shape :', arr1.shape)
print('ndim :', arr1.ndim)
```

```
[[1.  1.  1.  1.]
 [1.  1.  1.  1.]
 [1.  1.  1.  1.]]
size : 12
dtype : float64
shape : (3, 4)
ndim : 2
```

## 2.3 배열의 인덱싱과 슬라이싱

In [ ]:

```
import numpy as np

arr1 = np.array([[1,2,3],
                 [4,5,6],
                 [7,8,9]])

print(arr1[0])
print(arr1[0,2])
print(arr1[1:])
print(arr1[arr1 > 5])
```

```
[1 2 3]
3
[[4 5 6]
 [7 8 9]]
[6 7 8 9]
```

## 2.4 배열의 변형

In [ ]:

```
import numpy as np

arr = np.array([[1,2,3],[4,5,6]])

arr_T = arr.transpose()
arr_R = arr.reshape(3,2)
arr_F = arr.flatten()

print(arr)
print(arr_T)
print(arr_R)
print(arr_F)
```

```
[[1 2 3]
 [4 5 6]]
[[1 4]
 [2 5]
 [3 6]]
[[1 2]
 [3 4]
 [5 6]]
[1 2 3 4 5 6]
```

## 3.1 연산자에 의한 기본 연산

In [ ]:

```
import numpy as np

arr1 = np.arange(1,5)
arr2 = np.ones(4)

print(arr1, arr2)
print(arr1 + arr2)
print(arr1 - arr2)
print(arr1 > arr2)
```

```
[1 2 3 4] [1. 1. 1. 1.]
[2. 3. 4. 5.]
[0. 1. 2. 3.]
[False True True True]
```

In [ ]:

```
import numpy as np

arr1 = np.random.randint(11, size=4)

print(arr1)
print(arr1 + 2)
print(arr1 * 2)
```

```
[6 7 6 2]
[8 9 8 4]
[12 14 12 4]
```

## 3.2 유니버설 함수

In [ ]:

```
import numpy as np

arr1 = np.random.randint(-10, 11, size=5)

print(arr1)
print(np.min(arr1))
print(np.max(arr1))
print(np.sum(arr1))
print(np.mean(arr1))
print(np.var(arr1))
print(np.std(arr1))
```

```
[ 8 -10  2  1 -4]
-10
8
-3
-0.6
36.64
6.053098380168622
```

### 3.3 넘파이 배열 연산 연습

In [ ]:

```
import numpy as np

국어 = np.array([20,30,60])
영어 = np.array([60,40,90])

print(국어)
print(영어)
```

```
[20 30 60]
[60 40 90]
```

In [ ]:

```
총점 = 국어 + 영어
평균 = 총점/2

print(총점)
print(평균)
```

```
[ 80  70 150]
[40. 35. 75.]
```

In [ ]:

```
print(np.mean(국어))
```

```
36.666666666666664
```

## ※ 코랩에 나눔글꼴 설치하는 명령

In [ ]:

```
!sudo apt-get install -y fonts-nanum
```

```
!sudo fc-cache -fv
```

```
!rm ~/.cache/matplotlib -rf
```

Reading package lists... Done  
Building dependency tree  
Reading state information... Done  
The following package was automatically installed and is no longer required:  
libnvidia-common-460  
Use 'sudo apt autoremove' to remove it.  
The following NEW packages will be installed:  
fonts-nanum  
0 upgraded, 1 newly installed, 0 to remove and 49 not upgraded.  
Need to get 9,604 kB of archives.  
After this operation, 29.5 MB of additional disk space will be used.  
Get:1 <http://archive.ubuntu.com/ubuntu> (<http://archive.ubuntu.com/ubuntu>) bionic/universe amd64 fonts-nanum all 20170925-1 [9,604 kB]  
Fetched 9,604 kB in 4s (2,174 kB/s)  
debconf: unable to initialize frontend: Dialog  
debconf: (No usable dialog-like program is installed, so the dialog based frontend cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm line 76, <> line 1.)  
debconf: falling back to frontend: Readline  
debconf: unable to initialize frontend: Readline  
debconf: (This frontend requires a controlling tty.)  
debconf: falling back to frontend: Teletype  
dpkg-preconfigure: unable to re-open stdin:  
Selecting previously unselected package fonts-nanum.  
(Reading database ... 155653 files and directories currently installed.)  
Preparing to unpack .../fonts-nanum\_20170925-1\_all.deb ...  
Unpacking fonts-nanum (20170925-1) ...  
Setting up fonts-nanum (20170925-1) ...  
Processing triggers for fontconfig (2.12.6-0ubuntu2) ...  
/usr/share/fonts: caching, new cache contents: 0 fonts, 1 dirs  
/usr/share/fonts/truetype: caching, new cache contents: 0 fonts, 3 dirs  
/usr/share/fonts/truetype/humor-sans: caching, new cache contents: 1 fonts, 0 dirs  
/usr/share/fonts/truetype/liberation: caching, new cache contents: 16 fonts, 0 dirs  
/usr/share/fonts/truetype/nanum: caching, new cache contents: 10 fonts, 0 dirs  
/usr/local/share/fonts: caching, new cache contents: 0 fonts, 0 dirs  
/root/.local/share/fonts: skipping, no such directory  
/root/.fonts: skipping, no such directory  
/var/cache/fontconfig: cleaning cache directory  
/root/.cache/fontconfig: not cleaning non-existent cache directory  
/root/.fontconfig: not cleaning non-existent cache directory  
fc-cache: succeeded

